Maze Runner: EEE 121 Software Project 2

Sebastian Wolfgang Canillas
Electrical and Electronics Engineering Institute
University of the Philippines Diliman
Quezon City, Philippines
sebastian.wolfgang.canillas@eee.upd.edu.ph

***Abstract*—Data Structures and Algorithms are foundational to modern computing, enabling efficient data handling, optimization, and problem-solving across diverse applications. This course examines key data structures such as arrays, stacks, queues, linked lists, trees, and graphs, alongside core algorithmic techniques like sorting, searching, recursion, and dynamic programming. Efficient algorithm design is guided by complexity analysis, often evaluated using Big-O notation to understand performance implications [1]. Proper selection of data structures and algorithms can dramatically improve software scalability and efficiency, which is critical in fields such as artificial intelligence, web development, and database systems [2]. This paper documents the implementation of a maze generation system using Kruskal's algorithm. The system creates 2D rectangular mazes with exactly one unique path between any two points, implemented using C++ as the programming language. The documentation covers the design methodology, implementation details, and testing results.**

Keywords—C++, Data Structures and Algorithms, maze generation system, programming

I. INTRODUCTION

This paper serves as documentation for the 2nd software project in EEE 121. The software project simulates a Role-playing game (RPG) that includes mazes with unique layouts. The task is for the programmer to create a maze algorithm system that is to be done in two parts: (1) Maze Generation and (2) Determining the shortest path. This paper only covers the first part, as the software project was done individually and the second part is not to be done. The algorithm treats the maze as a grid of cells with potential walls between them and constructs a minimum spanning tree to ensure a perfect maze, a configuration with no cycles and exactly one path between any two points. The use of union-find allows for efficient cycle detection and merging of components during the wall-removal process.

II. METHODOLOGY

A. Integrated Development Environment (IDE)

While Visual Studio Code (VS Code) is more commonly used, the programmer used Visual Studio Community 2022 as the IDE to program the software project. This is due to the latter having an easier compiler to set-up. Because of this, the program does not use the suggested method to integrate and run the .cpp files.

B. Grid Initialization and Representation

The program accepts two integer inputs which represent the number of rows and columns of the grid. The grid is stored using two two-dimensional boolean vectors: one to represent the right walls of each cell and another to represent the bottom walls. Initially, every wall is set to true, indicating that all cells are enclosed from all directions.

```
class Maze { // Derived from the specs given code snippet
private:
    int M, N; // M for rows, N for columns
    std::vector<std::vector<bool>> right_walls; // Right walls of each cell
    std::vector<std::vector<bool>> down_walls;  // Down walls of each cell
    std::vector<int> parent; // For disjoint-set
```

Fig. 1. Code lines for Grid Initialization

C. Edge Generation and Randomization

All potential edges are defined as walls that are removable between adjacent tiles. The program considers two types of adjacencies: Horizontal (i.e. a tile is connected to one that is immediately to its right) and Vertical (i.e. a tile is connected to one that is immediately below it).

For each of these valid neighbor relationships, a corresponding edge is added to a list of candidate edges. These edges represent the walls that could potentially be removed to link neighboring cells.

To ensure randomness in generated mazes, the list of edges is randomly shuffled using the std::shuffle() function.

```
void generateMaze() {
    // Generate all possible edges (walls that can be removed)
    std::vector<std::pair<int, int>> edges;

    // Right walls
    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < N - 1; ++j) {
            edges.emplace_back(toIndex(i, j), toIndex(i, j + 1));
        }
    }

    // Down walls
    for (int i = 0; i < M - 1; ++i) {
        for (int j = 0; j < N; ++j) {
            edges.emplace_back(toIndex(i, j), toIndex(i + 1, j));
        }
    }

    // Random edges
    std::shuffle(edges.begin(), edges.end(), std::default_random_engine(std::time(0)));
```

Fig. 2. Code lines for Edge Generation and Randomization

D. *Disjoint-Set Operations and Maze Generation (Kruskal's Algorithm)*

The maze generation system was implemented using Kruskal's algorithm, which constructs a perfect maze by treating each grid cell as a node and potential wall removals as edges in a graph. Initially, all cells are isolated with intact walls. The algorithm then processes a randomly shuffled list of all possible walls, removing those that connect disjoint sets of cells while ensuring no cycles form.

Vertical and horizontal walls are tracked separately in boolean matrices, with removals determined by whether adjacent cells belong to different sets. The resulting maze is visualized in ASCII, where walls ("|", "---") and passages (" ") are rendered systematically. This approach guarantees a single unique path between any two cells for an M×N grid.

```
// Implementation of Kruskal's algorithm
for (const auto& edge : edges) {
    int u = edge.first;
    int v = edge.second;

    if (find(u) != find(v)) {
        unionSets(u, v);

        // Remove the wall between u and v
        int u_row = u / N, u_col = u % N;
        int v_row = v / N, v_col = v % N;

        if (u_row == v_row) {
            // Right wall
            right_walls[u_row][u_col] = false;
        }
        else {
            // Down wall
            down_walls[u_row][u_col] = false;
        }
    }
}
```

Fig. 3. Code lines for Disjoint-Set Operations and Maze Generation

E. *Maze Visualization and Output*

After the maze has been constructed, it is printed to the console in ASCII format. The generateGrid() method first prints the initial enclosed grid. The generateMaze() method then constructs and displays the final maze, where walls are removed to form a maze with a single, unique path.

Each tile is represented by three spaces " ", and vertical bars (|) and plus-minus combinations (+---) are used to depict walls. This visual output follows the sample I/O in the project specs.

```
void printMaze() {
    for (int i = 0; i < M; ++i) {
        // Top border
        for (int j = 0; j < N; ++j) {
            std::cout << "+";
            std::cout << (i == 0 ? "---" : (down_walls[i - 1][j] ? "---" : "   "));
        }
        std::cout << "+\n";

        // Cells with right walls
        std::cout << "|";
        for (int j = 0; j < N; ++j) {
            std::cout << "   ";
            if (j < N - 1) {
                std::cout << (right_walls[i][j] ? "|" : " ");
            }
            else {
                std::cout << "|";
            }
        }
        std::cout << "\n";
    }

    // Bottom border
    for (int j = 0; j < N; ++j) {
        std::cout << "+---";
    }
    std::cout << "+\n";
}
```

Fig. 4. Code lines for Maze Visualization and Output

III. Conclusion

This project successfully implemented a randomized perfect maze generator using Kruskal's algorithm. The resulting maze satisfies requirements laid out in the specs sheet. The input and output is also similar to that given as an example. The disjoint-set operations effectively supported the MST logic, and the system provides a strong base for subsequent implementation of shortest-path algorithms.

Acknowledgment

Special thanks to Ma'am Issa Tuso for the wonderful semester. Made the rather daunting course understandable through activities that made her students truly appreciate the wonders of Data Structures and Algorithms.

References

The following references were used in the Abstract part of this paper:

[1] GeeksforGeeks. (2023). *Fundamentals of Algorithms*. Retrieved from https://www.geeksforgeeks.org/fundamentals-of-algorithms/

[2] Khan Academy. (n.d.). *Algorithms*. Retrieved from https://www.khanacademy.org/computing/computer-science/algorithms